Day 7: Pandas — Part 1

AI and Machine Learning Course

Sandesh Dhital

Nesfield International College

July 28, 2026

Today's Agenda

1. Introduction to Pandas
2. Series and DataFrames
3. Reading CSV/Excel Files
4. Selecting, Filtering, and Sorting
5. Practice: Explore a Real Dataset

What is Pandas?

- **Pandas** = Panel Data / Python Data Analysis Library
- A Python library for working with **tabular data** (rows and columns)
- Built on top of NumPy — uses NumPy arrays internally
- The go-to tool for data manipulation and analysis in Python

Why Pandas instead of NumPy?

- NumPy is great for numbers, but real data has **mixed types** (names, dates, numbers)
- Pandas gives you **labeled** rows and columns (like a spreadsheet)
- Built-in tools for reading CSV/Excel, handling missing data, grouping, and more

Installing and Importing Pandas

- Install with: `pip install pandas`
- Import as: `import pandas as pd`
- Everyone uses the short name `pd` — follow this convention

Already Installed?

If you installed Anaconda, Pandas is already available. Just import it.

Series — A Single Column of Data

- A **Series** is a 1D labeled array — like a single column in a spreadsheet
- Each element has a **label** (index) and a **value**
- Can hold any data type: integers, floats, strings, etc.

Creating a Series

Method	What it does
`pd.Series([...])`	From a Python list
`pd.Series({...})`	From a dictionary (keys become index)
`pd.Series(value, index=[...])`	Scalar repeated with custom index

Key Properties

`.values`, `.index`, `.dtype`, `.shape`, `.name`

DataFrame — A Table of Data

- A **DataFrame** is a 2D labeled data structure — like a spreadsheet or SQL table
- Has **rows** (index) and **columns** (each column is a Series)
- Columns can have **different data types**

Creating a DataFrame

Method	What it does
`pd.DataFrame({...})`	From a dictionary of lists
`pd.DataFrame(list of dicts)`	From a list of dictionaries
`pd.DataFrame(numpy_array)`	From a NumPy array
`pd.read_csv("file.csv")`	From a CSV file
`pd.read_excel("file.xlsx")`	From an Excel file

DataFrame Properties

Property / Method	What it returns	Example
`.shape`	(rows, columns)	(150, 4)
`.columns`	Column names	Index(['Name', 'Age', ...])
`.dtypes`	Data type of each column	Name: object, Age: int64
`.index`	Row labels	RangeIndex(0, 150)
`.head(n)`	First *n* rows (default 5)	Top 5 rows
`.tail(n)`	Last *n* rows	Bottom 5 rows
`.info()`	Summary (types, non-null counts)	Column info
`.describe()`	Statistics (mean, std, min, max)	Numerical summary

Reading Data from Files

- Most real data comes in **CSV** (Comma-Separated Values) or **Excel** files
- Pandas makes reading them a one-liner

Reading Functions

Function	What it reads
`pd.read_csv("file.csv")`	CSV files
`pd.read_excel("file.xlsx")`	Excel files
`pd.read_json("file.json")`	JSON files
`pd.read_html("url")`	Tables from web pages

Useful Parameters for `read_csv`

- `sep=","` — delimiter (default is comma)
- `header=0` — which row is the header

Writing Data to Files

- After processing data, you can **save** your DataFrame back to a file

Writing Functions

Method	What it writes
`df.to_csv("output.csv")`	CSV file
`df.to_excel("output.xlsx")`	Excel file
`df.to_json("output.json")`	JSON file

Tip

Use `index=False` to avoid saving the row numbers as a column:

```
df.to_csv("output.csv", index=False)
```

Selecting Columns

- **Single column**: `df["column_name"]` — returns a Series
- **Multiple columns**: `df[["col1", "col2"]]` — returns a DataFrame
- **Dot notation**: `df.column_name` — works if column name has no spaces

`loc` vs `iloc`

Method	Selects by	Example
`df.loc[row, col]`	Labels (names)	`df.loc[0, "Name"]`
`df.iloc[row, col]`	Integer position	`df.iloc[0, 0]`

Remember

`loc` includes the end point. `iloc` excludes it (like Python slicing).

Filtering Rows

- Apply a **condition** to get only the rows you want
- Works like NumPy boolean indexing

Examples

- `df[df["Age"] > 30]` — rows where Age is greater than 30
- `df[df["City"] == "Kathmandu"]` — rows where City is Kathmandu
- `df[(df["Age"] > 20) & (df["Age"] < 30)]` — combine with `&` (AND)
- `df[(df["City"] == "Pokhara") | (df["City"] == "Lalitpur")]` — combine with `|` (OR)

Important

Always wrap each condition in parentheses when combining with `&` or `|`.

More Filtering Methods

Method	What it does
`df["col"].isin([...])`	Check if value is in a list
`df["col"].between(a, b)`	Check if value is in range $[a, b]$
`df["col"].str.contains("text")`	Check if string contains text
`df["col"].str.startswith("A")`	Check if string starts with "A"
`df["col"].isna()`	Check for missing values
`df["col"].notna()`	Check for non-missing values
`df.query("Age > 30")`	Filter using a string expression

Sorting Data

- **Sort by values**: `df.sort_values("column")`
- **Sort descending**: `df.sort_values("column", ascending=False)`
- **Sort by multiple columns**: `df.sort_values(["col1", "col2"])`
- **Sort by index**: `df.sort_index()`

Ranking

- `df["col"].rank()` — assigns ranks (1, 2, 3, ...) based on values
- `df.nlargest(5, "col")` — top 5 rows by a column
- `df.nsmallest(5, "col")` — bottom 5 rows by a column

Adding and Modifying Columns

- **Add a new column**: `df["new_col"] = values`
- **Modify existing**: `df["col"] = df["col"] * 2`
- **Computed column**: `df["total"] = df["math"] + df["science"]`
- **Apply a function**: `df["col"].apply(func)`
- **Drop a column**: `df.drop(columns=["col"])`
- **Rename columns**: `df.rename(columns={"old": "new"})`

In-place vs Copy

Most Pandas methods return a **new DataFrame** by default. Use `inplace=True` to modify the original, or reassign: `df = df.drop(columns=["col"])`

Practice Exercises

1. Create a DataFrame of 5 students with columns: Name, Age, City, Score.
2. Read a CSV file into a DataFrame. Display the first 5 rows, shape, and info.
3. Select only the "Name" and "Score" columns.
4. Filter students with Score above 80.
5. Sort the DataFrame by Score in descending order.
6. Add a new column "Grade" based on Score (A: $\geq$90, B: $\geq$80, C: $\geq$70, else F).
7. Use the Iris dataset: find the mean sepal length for each species.

Summary

- **Series**: A single labeled column of data
- **DataFrame**: A labeled table (rows $\times$ columns) — the core Pandas structure
- **Reading files**: `pd.read_csv()`, `pd.read_excel()` — one-liner data loading
- **Selecting**: `df["col"]`, `df.loc[]`, `df.iloc[]` — pick rows/columns
- **Filtering**: Boolean conditions to keep only the rows you need
- **Sorting**: `df.sort_values()` — order your data
- **Modifying**: Add, rename, drop, or transform columns

Next: Day 8 — Pandas Part 2

Handling missing data, GroupBy, aggregation, pivot tables, merging DataFrames